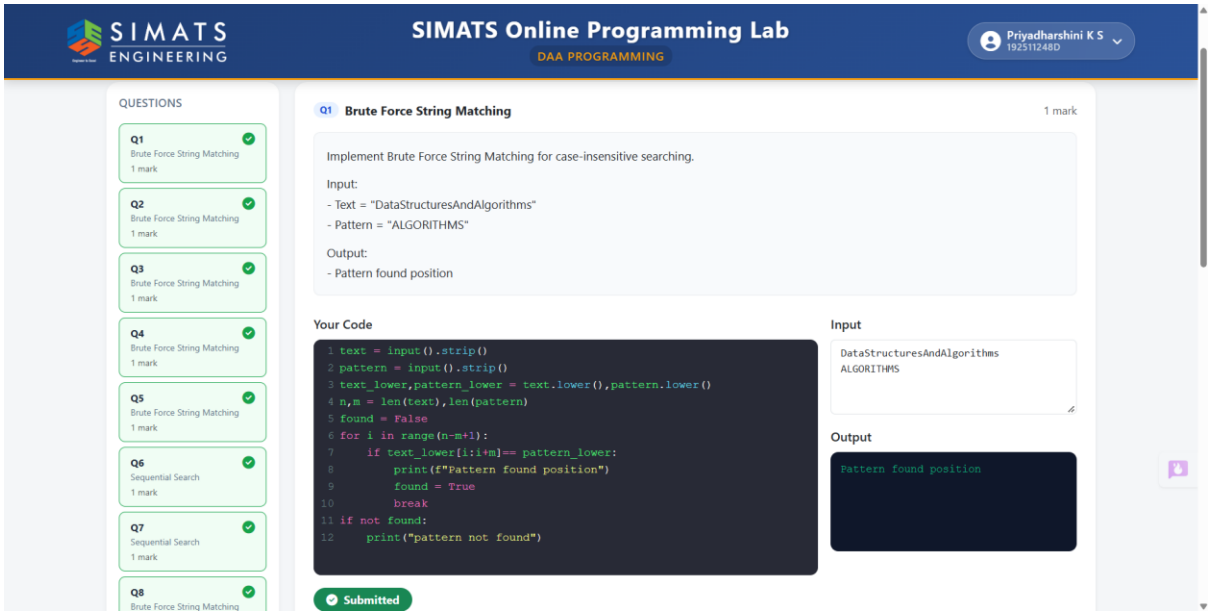
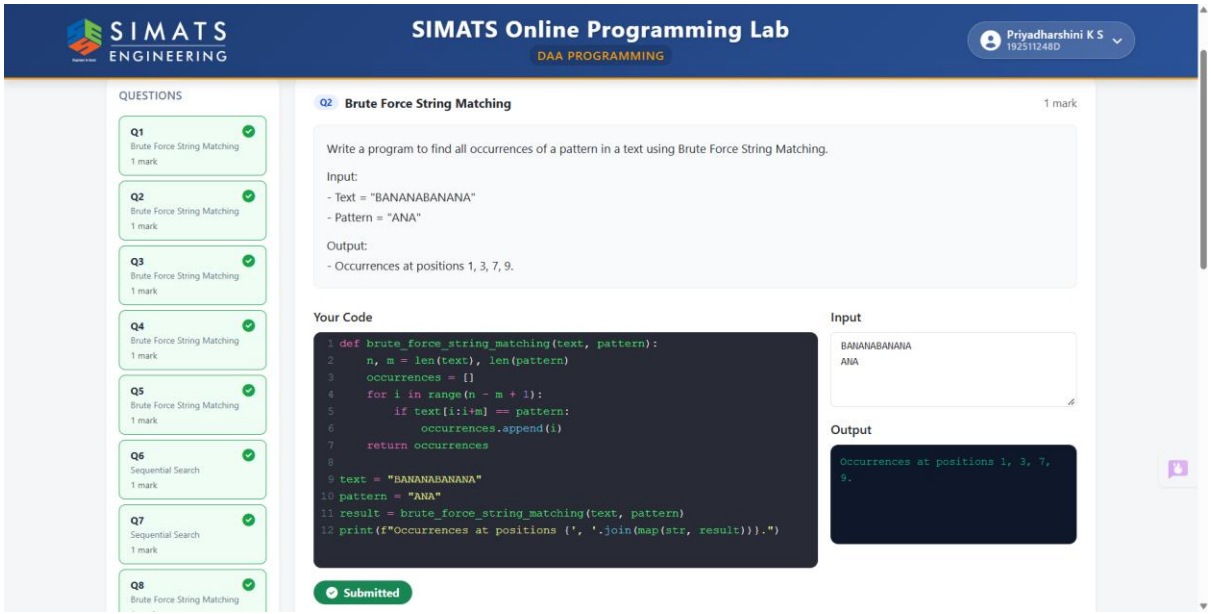
LAB PROGRAM - TOPIC 3

1. BRUTE FORCE TECHNIQUE





SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Priyadharshini K S
192511248D

QUESTIONS

- Q1 Brute Force String Matching 1 mark
- Q2 Brute Force String Matching 1 mark
- Q3 Brute Force String Matching 1 mark
- Q4 Brute Force String Matching 1 mark
- Q5 Brute Force String Matching 1 mark
- Q6 Sequential Search 1 mark
- Q7 Sequential Search 1 mark
- Q8 Brute Force String Matching

Q3 Brute Force String Matching — 1 mark

Implement Brute Force String Matching and display the comparison table for each alignment.

Input:
- Text = "MISSISSIPPI"
- Pattern = "ISSI"

Output:
- Shift number
- Comparisons made
- Match/Mismatch result

Your Code

```
def brute_force_string_matching(text, pattern):
    n, m = len(text), len(pattern)
    print(f"{'Shift':<10} {'Comparisons':<15} {'Result':<15}")
    for s in range(n - m + 1):
        comparisons = 0
        match = True
        for j in range(m):
            comparisons += 1
            if text[s + j] != pattern[j]:
                match = False
                break
        result = "Match" if match else "Mismatch"
        print(f"{s:<10} {comparisons:<15} {result:<15}")

```

Input

```
text = "MISSISSIPPI"
pattern = "ISSI"
```

Output

```
Shift      Comparisons     Result
0          1
Mismatch
1          4               Match
2          1
Mismatch
```

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Priyadharshini K S
192511248D

QUESTIONS

- Q1 Brute Force String Matching 1 mark
- Q2 Brute Force String Matching 1 mark
- Q3 Brute Force String Matching 1 mark
- Q4 Brute Force String Matching 1 mark
- Q5 Brute Force String Matching 1 mark
- Q6 Sequential Search 1 mark
- Q7 Sequential Search 1 mark
- Q8 Brute Force String Matching

Q4 Brute Force String Matching — 1 mark

Implement Brute Force String Matching and determine whether the search is:
- Best Case
- Average Case
- Worst Case

Input:
- Text = "AAAAAAAAAB"
- Pattern = "AAAAB"

Display the number of comparisons performed.

Your Code

```
def brute_force(text, pattern):
    n, m = len(text), len(pattern)
    comparisons = 0
    for i in range(n - m + 1):
        for j in range(m):
            comparisons += 1
            if text[i + j] != pattern[j]:
                break
        else:
            break
    print(comparisons)

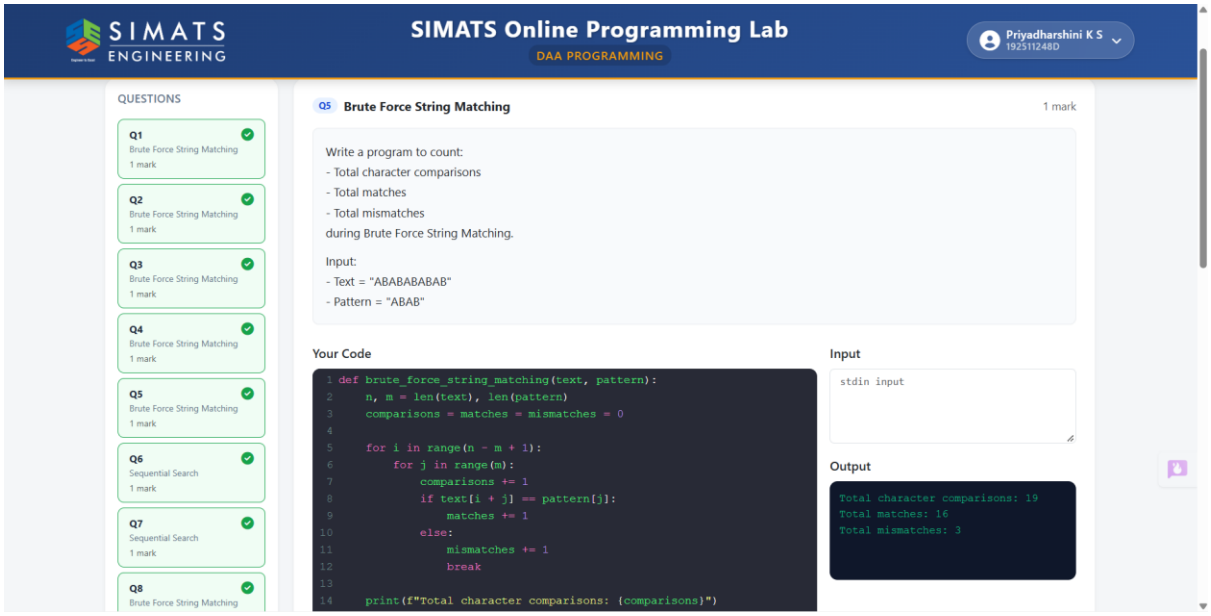
text = "AAAAAAAAAB"
pattern = "AAAAB"
```

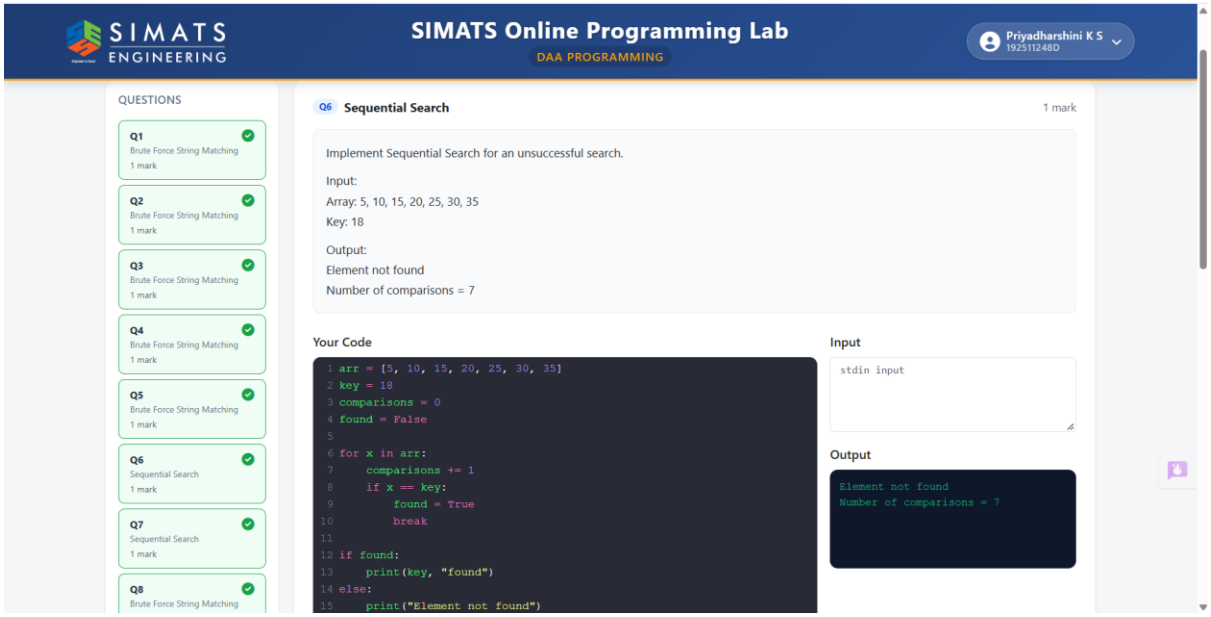
Input

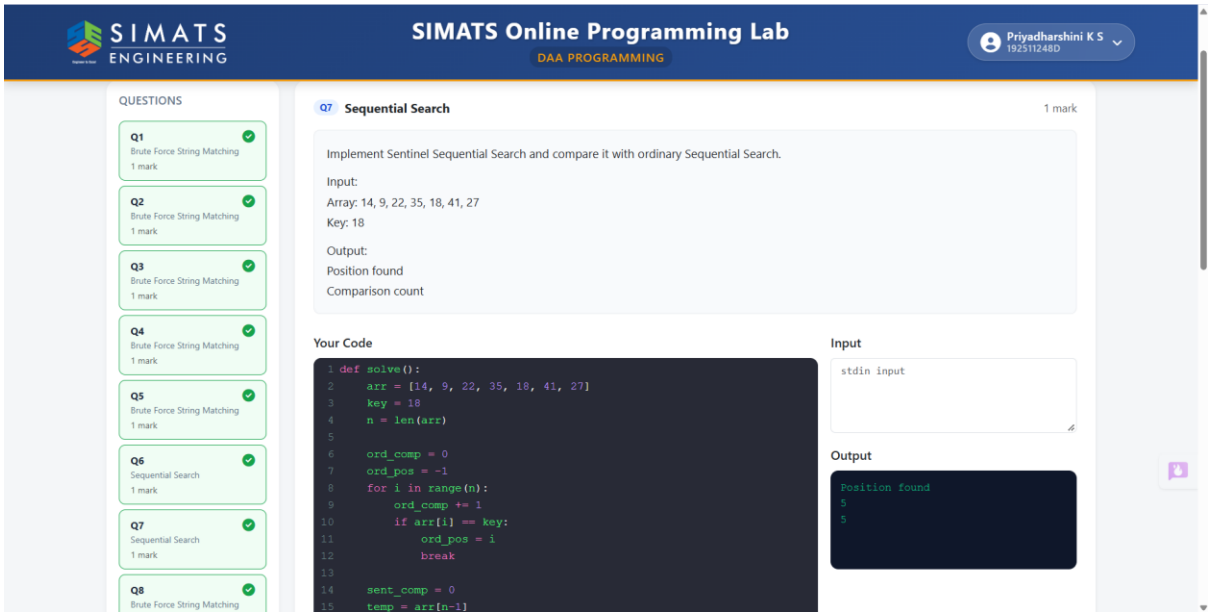
```
stdin input
```

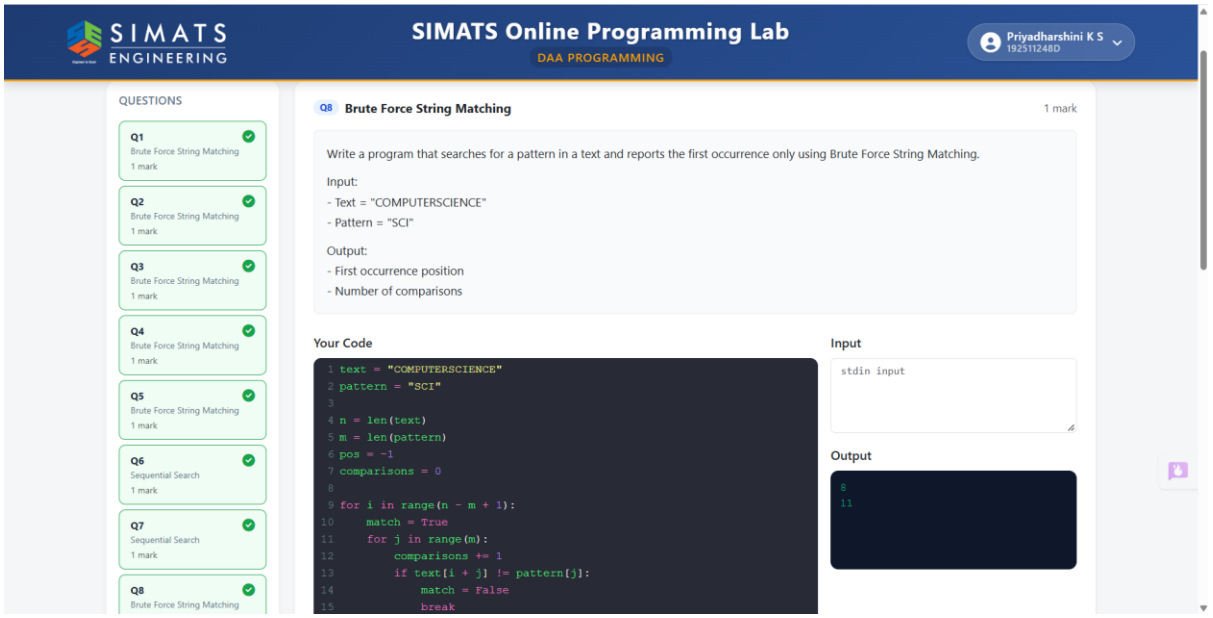
Output

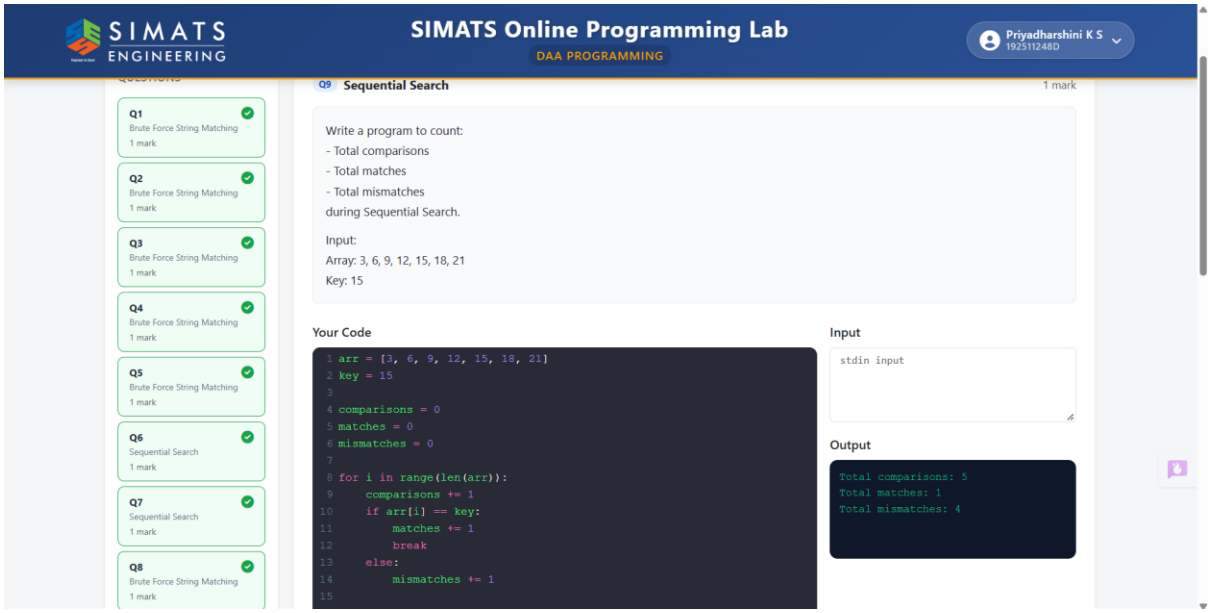
```
30
```

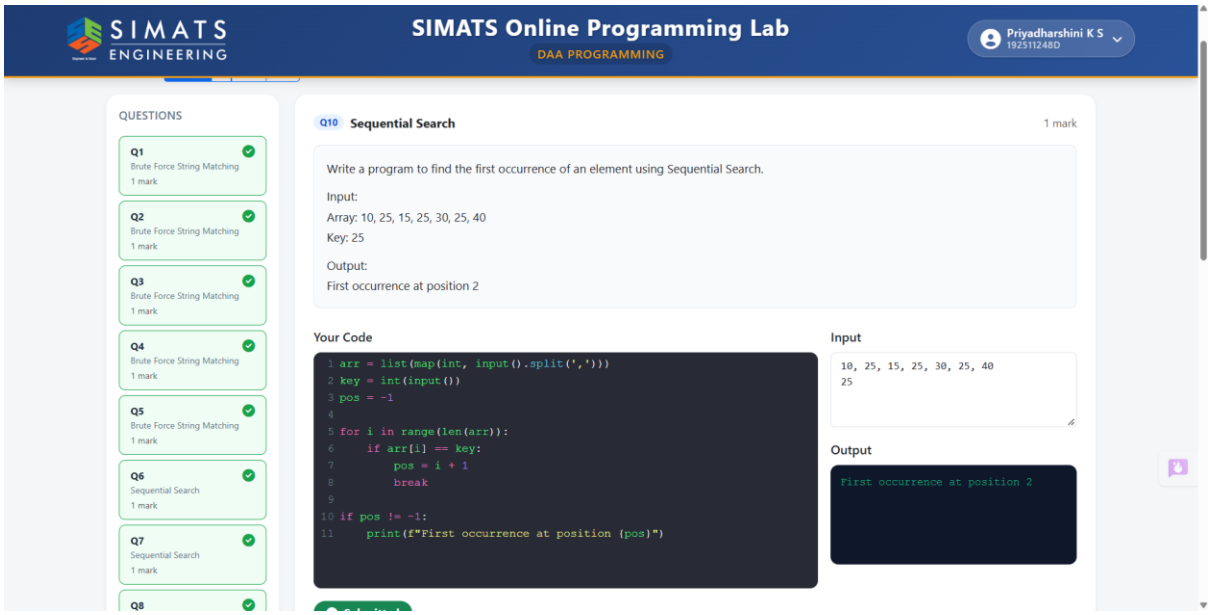

SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
QUESTIONS
Q1 Brute Force String Matching 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Brute Force String Matching
Q5 Brute Force String Matching
1 mark
Write a program to count:
- Total character comparisons
- Total matches
- Total mismatches
during Brute Force String Matching.
Input:
- Text = "ABABABABAB"
- Pattern = "ABAB"
Your Code
def brute_force_string_matching(text, pattern):
n, m = len(text), len(pattern)
comparisons = matches = mismatches = 0
for i in range(n - m + 1):
for j in range(m):
comparisons += 1
if text[i + j] == pattern[j]:
matches += 1
else:
mismatches += 1
break
print(f"Total character comparisons: {comparisons}")
Input
stdin input
Output
Total character comparisons: 19
Total matches: 16
Total mismatches: 3


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
QUESTIONS
Q1 Brute Force String Matching 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Brute Force String Matching
Q6 Sequential Search
1 mark
Implement Sequential Search for an unsuccessful search.
Input:
Array: 5, 10, 15, 20, 25, 30, 35
Key: 18
Output:
Element not found
Number of comparisons = 7
Your Code
arr = [5, 10, 15, 20, 25, 30, 35]
key = 18
comparisons = 0
found = False
for x in arr:
comparisons += 1
if x == key:
found = True
break
if found:
print(key, "found")
else:
print("Element not found")
Input
stdin input
Output
Element not found
Number of comparisons = 7

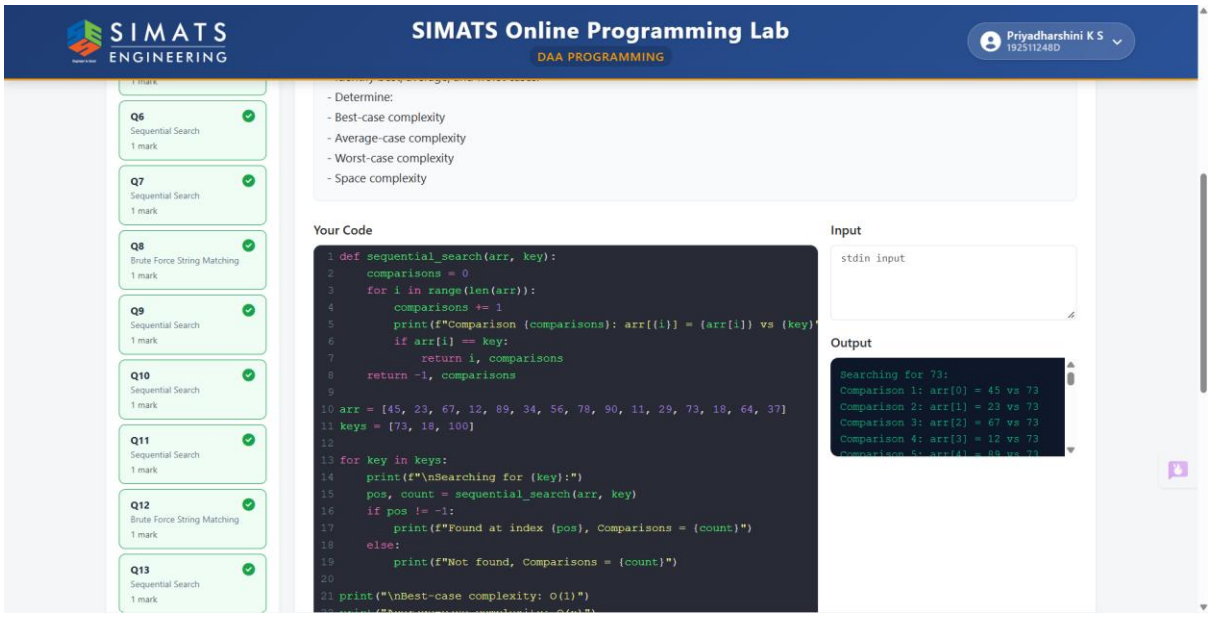

SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
QUESTIONS
Q1 Brute Force String Matching 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Brute Force String Matching
Q7 Sequential Search
1 mark
Implement Sentinel Sequential Search and compare it with ordinary Sequential Search.
Input:
Array: 14, 9, 22, 35, 18, 41, 27
Key: 18
Output:
Position found
Comparison count
Your Code
def solve():
arr = [14, 9, 22, 35, 18, 41, 27]
key = 18
n = len(arr)
ord_comp = 0
ord_pos = -1
for i in range(n):
ord_comp += 1
if arr[i] == key:
ord_pos = i
break
sent_comp = 0
temp = arr[n-1]
Input
stdin input
Output
Position found
5
5


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
QUESTIONS
Q1 Brute Force String Matching 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Brute Force String Matching
Q8 Brute Force String Matching
1 mark
Write a program that searches for a pattern in a text and reports the first occurrence only using Brute Force String Matching.
Input:
- Text = "COMPUTERSCIENCE"
- Pattern = "SCI"
Output:
- First occurrence position
- Number of comparisons
Your Code
text = "COMPUTERSCIENCE"
pattern = "SCI"
n = len(text)
m = len(pattern)
pos = -1
comparisons = 0
for i in range(n - m + 1):
match = True
for j in range(m):
comparisons += 1
if text[i + j] != pattern[j]:
match = False
break
Input
stdin input
Output
8
11


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
Q1 Brute Force String Matching 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Brute Force String Matching 1 mark
Q9 Sequential Search
1 mark
Write a program to count:
- Total comparisons
- Total matches
- Total mismatches
during Sequential Search.
Input:
Array: 3, 6, 9, 12, 15, 18, 21
Key: 15
Your Code
arr = [3, 6, 9, 12, 15, 18, 21]
key = 15
comparisons = 0
matches = 0
mismatches = 0
for i in range(len(arr)):
comparisons += 1
if arr[i] == key:
matches += 1
break
else:
mismatches += 1
Input
stdin input
Output
Total comparisons: 5
Total matches: 1
Total mismatches: 4


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
QUESTIONS
Q1 Brute Force String Matching 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8
Q10 Sequential Search
1 mark
Write a program to find the first occurrence of an element using Sequential Search.
Input:
Array: 10, 25, 15, 25, 30, 25, 40
Key: 25
Output:
First occurrence at position 2
Your Code
arr = list(map(int, input().split(',')))
key = int(input())
pos = -1
for i in range(len(arr)):
if arr[i] == key:
pos = i + 1
break
if pos != -1:
print(f"First occurrence at position {pos}")
Input
10, 25, 15, 25, 30, 25, 40
25
Output
First occurrence at position 2

SEQUENTIAL SEARCH

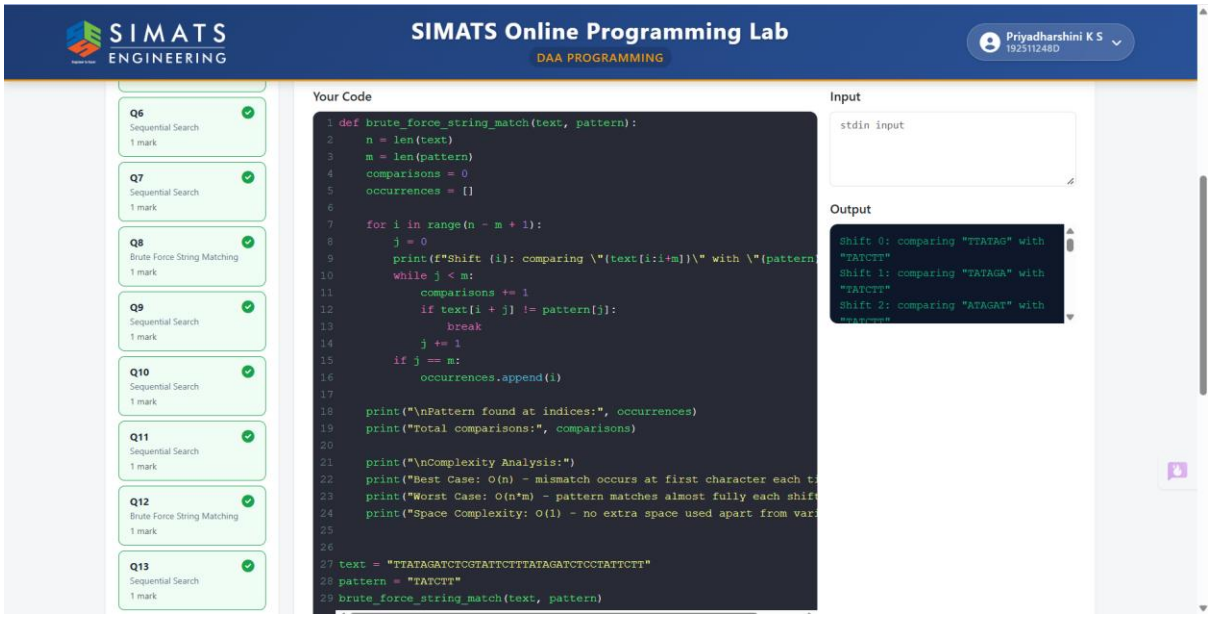



```
def sequential_search(arr, key):
    comparisons = 0
    for i in range(len(arr)):
        comparisons += 1
        print(f"Comparison {comparisons}: arr[{i}] = {arr[i]} vs {key}")
        if arr[i] == key:
            return i, comparisons
    return -1, comparisons

arr = [45, 23, 67, 12, 89, 34, 56, 78, 90, 11, 29, 73, 18, 64, 37]
keys = [73, 18, 100]

for key in keys:
    print(f"\nSearching for {key}:")
    pos, count = sequential_search(arr, key)
    if pos != -1:
        print(f"Found at index {pos}, Comparisons = {count}")
    else:
        print(f"Not found, Comparisons = {count}")

print("\nBest-case complexity: O(1)")
```




```
def brute_force_string_match(text, pattern):
    n = len(text)
    m = len(pattern)
    comparisons = 0
    occurrences = []

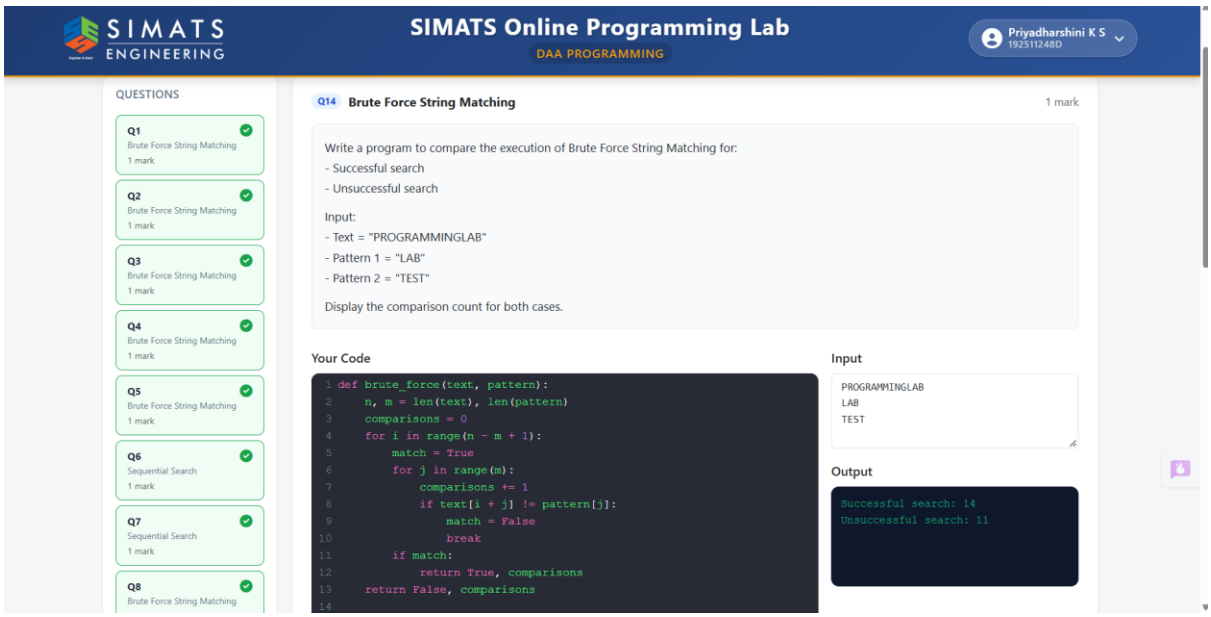
    for i in range(n - m + 1):
        j = 0
        print(f"Shift {i}: comparing \"{text[i:i+m]}\" with \"{pattern}\"")
        while j < m:
            comparisons += 1
            if text[i + j] != pattern[j]:
                break
            j += 1
        if j == m:
            occurrences.append(i)

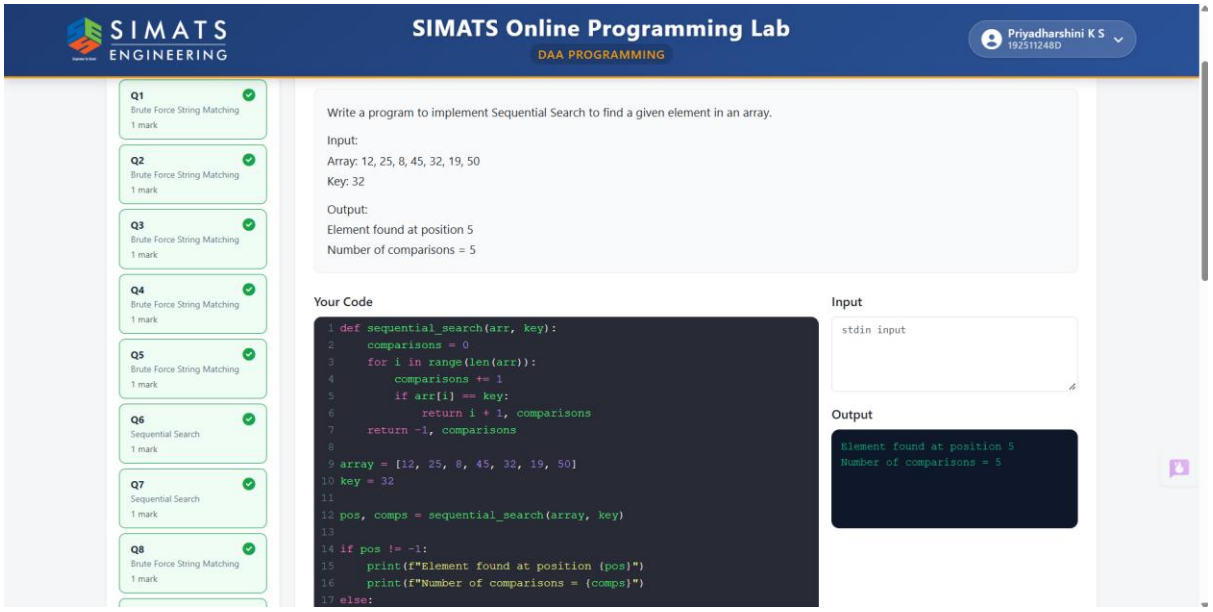
    print("\nPattern found at indices:", occurrences)
    print("Total comparisons:", comparisons)

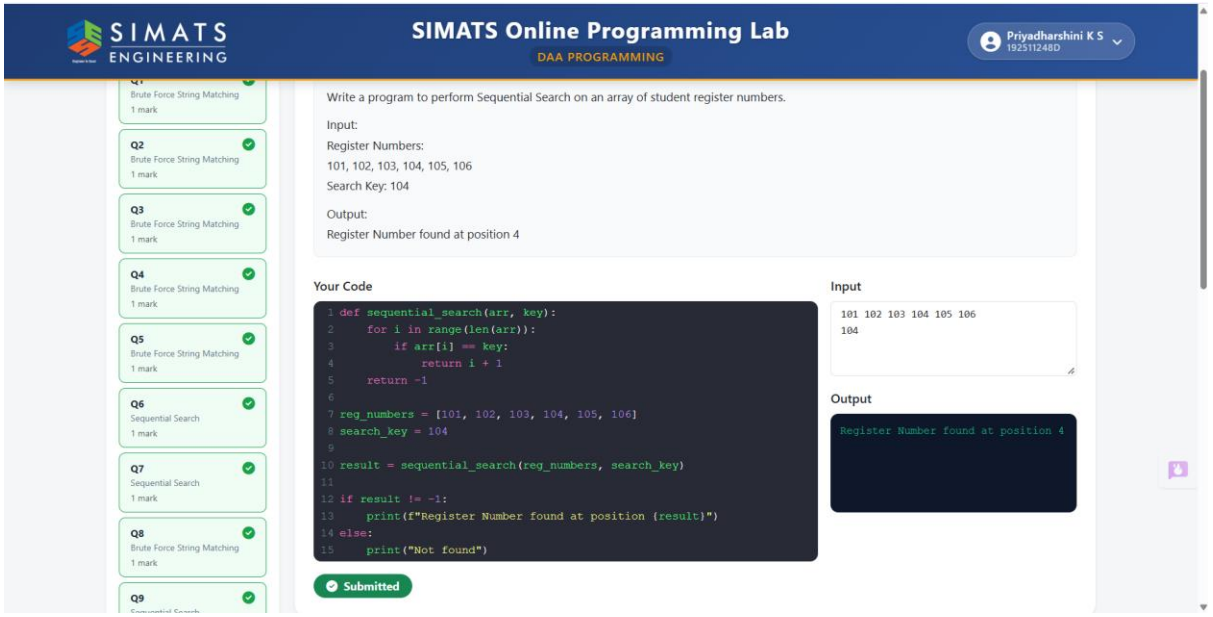
    print("\nComplexity Analysis:")
    print("Best Case: O(n) - mismatch occurs at first character each ti
    print("Worst Case: O(n*m) - pattern matches almost fully each shift
    print("Space Complexity: O(1) - no extra space used apart from vari

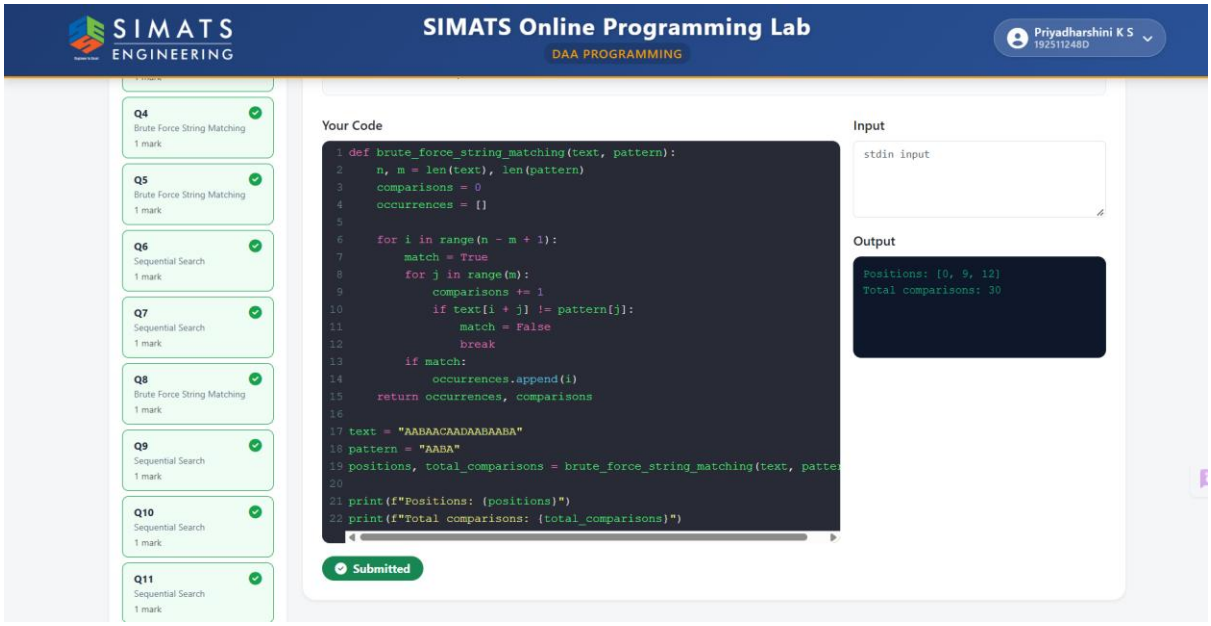

text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT"
pattern = "TATCTT"
brute_force_string_match(text, pattern)
```


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
Brute Force String Matching
1 mark
Q2
Brute Force String Matching
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Brute Force String Matching
1 mark
Q9
Write a program to perform Sequential Search on a 2-D matrix.
Input:
12 8 15
5 18 27
9 11 24
Key = 24
Output:
Element found at Row 3 Column 3
Your Code
matrix = []
for _ in range(3):
matrix.append(list(map(int, input().split())))
key = int(input())
found = False
for i in range(len(matrix)):
for j in range(len(matrix[i])):
if matrix[i][j] == key:
print(f"Element found at Row {i+1} Column {j+1}")
found = True
break
if found:
break
Input
12 8 15
5 18 27
9 11 24
24
Output
Element found at Row 3 Column 3

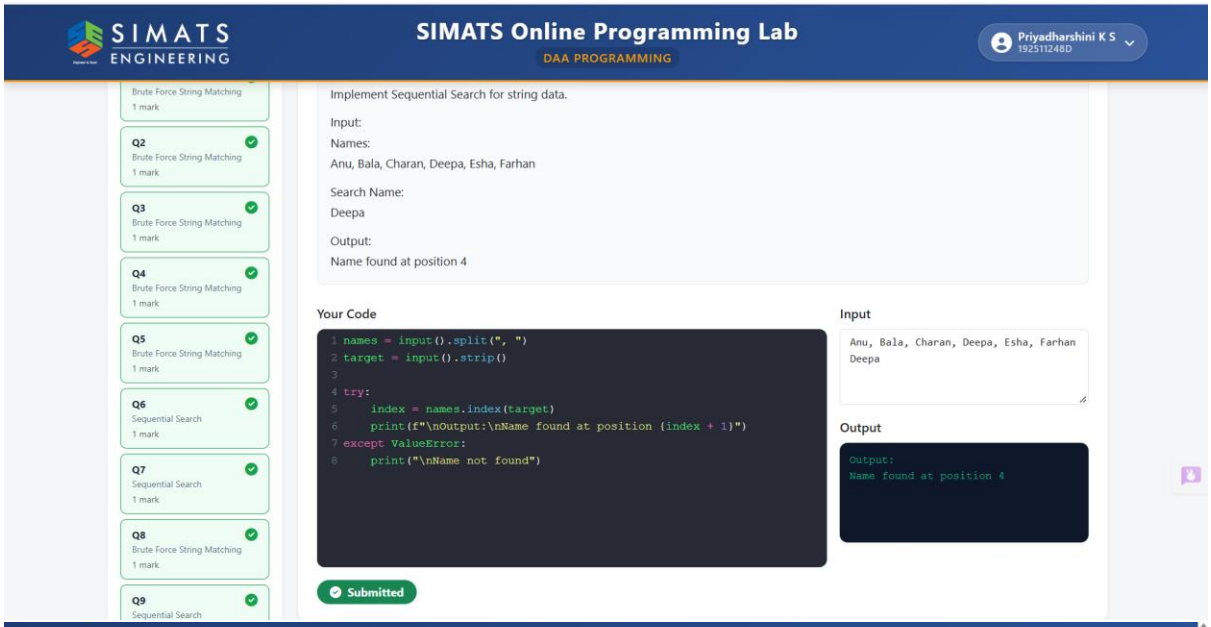

SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
QUESTIONS
Q1
Brute Force String Matching
1 mark
Q2
Brute Force String Matching
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Brute Force String Matching
Q14 Brute Force String Matching
1 mark
Write a program to compare the execution of Brute Force String Matching for:
- Successful search
- Unsuccessful search
Input:
- Text = "PROGRAMMINGLAB"
- Pattern 1 = "LAB"
- Pattern 2 = "TEST"
Display the comparison count for both cases.
Your Code
def brute_force(text, pattern):
n, m = len(text), len(pattern)
comparisons = 0
for i in range(n - m + 1):
match = True
for j in range(m):
comparisons += 1
if text[i + j] != pattern[j]:
match = False
break
if match:
return True, comparisons
return False, comparisons
Input
PROGRAMMINGLAB
LAB
TEST
Output
Successful search: 14
Unsuccessful search: 11

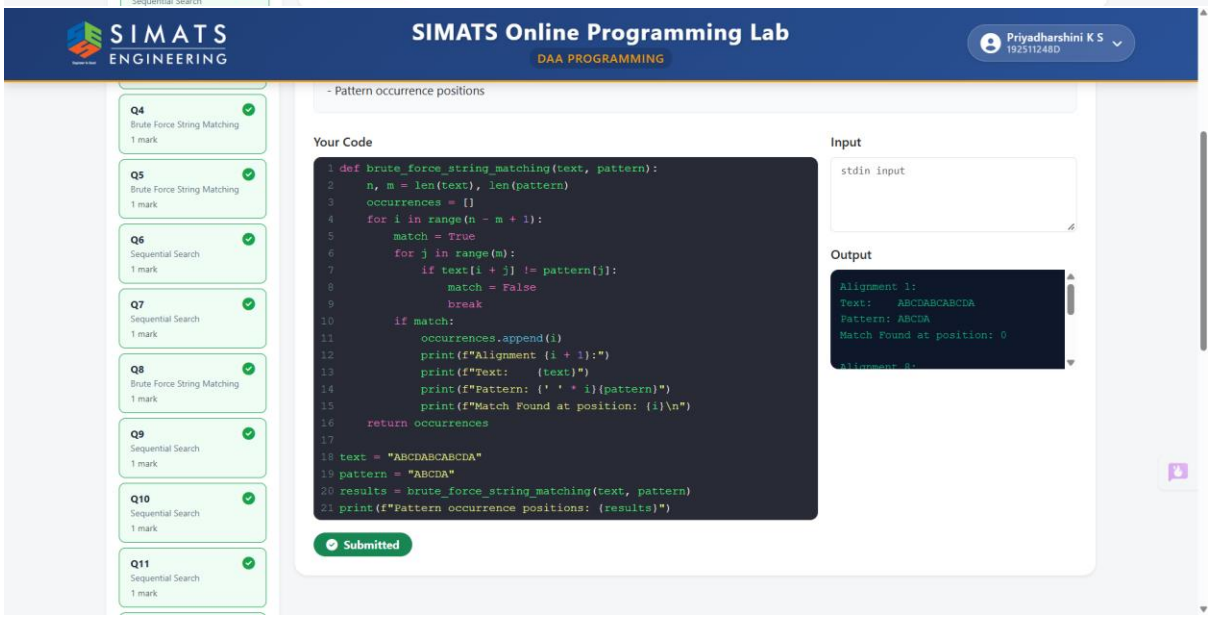

SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
Q1 Brute Force String Matching 1 mark
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Brute Force String Matching 1 mark
Write a program to implement Sequential Search to find a given element in an array.
Input:
Array: 12, 25, 8, 45, 32, 19, 50
Key: 32
Output:
Element found at position 5
Number of comparisons = 5
Your Code
def sequential_search(arr, key):
comparisons = 0
for i in range(len(arr)):
comparisons += 1
if arr[i] == key:
return i + 1, comparisons
return -1, comparisons
array = [12, 25, 8, 45, 32, 19, 50]
key = 32
pos, comps = sequential_search(array, key)
if pos != -1:
print(f"Element found at position {pos}")
print(f"Number of comparisons = {comps}")
else:
Input
stdin input
Output
Element found at position 5
Number of comparisons = 5


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
Q2 Brute Force String Matching 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Brute Force String Matching 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Sequential Search 1 mark
Q7 Sequential Search 1 mark
Q8 Brute Force String Matching 1 mark
Q9
Write a program to perform Sequential Search on an array of student register numbers.
Input:
Register Numbers:
101, 102, 103, 104, 105, 106
Search Key: 104
Output:
Register Number found at position 4
Your Code
def sequential_search(arr, key):
for i in range(len(arr)):
if arr[i] == key:
return i + 1
return -1
reg_numbers = [101, 102, 103, 104, 105, 106]
search_key = 104
result = sequential_search(reg_numbers, search_key)
if result != -1:
print(f"Register Number found at position {result}")
else:
print("Not found")
Submitted
Input
101 102 103 104 105 106
104
Output
Register Number found at position 4


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Brute Force String Matching
1 mark
Q9
Sequential Search
1 mark
Q10
Sequential Search
1 mark
Q11
Sequential Search
1 mark
Your Code
def brute_force_string_matching(text, pattern):
n, m = len(text), len(pattern)
comparisons = 0
occurrences = []
for i in range(n - m + 1):
match = True
for j in range(m):
comparisons += 1
if text[i + j] != pattern[j]:
match = False
break
if match:
occurrences.append(i)
return occurrences, comparisons
text = "AABAACAADAABAABA"
pattern = "AABA"
positions, total_comparisons = brute_force_string_matching(text, patte
print(f"Positions: {positions}")
print(f"Total comparisons: {total_comparisons}")
Submitted
Input
stdin input
Output
Positions: [0, 9, 12]
Total comparisons: 30


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
Q1
Brute Force String Matching
1 mark
Q2
Brute Force String Matching
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Brute Force String Matching
1 mark
Implement Sequential Search to find all occurrences of a given element.
Input:
Array: 7, 12, 7, 25, 18, 7, 30, 7
Key: 7
Output:
Occurrences at positions:
1, 3, 6, 8
Total occurrences = 4
Your Code
arr = [7, 12, 7, 25, 18, 7, 30, 7]
key = 7
pos = [i + 1 for i, val in enumerate(arr) if val == key]
print("Occurrences at positions:")
print(*pos, sep=", ")
print(f"Total occurrences = {len(pos)}")
Input
stdin input
Output
Occurrences at positions:
1, 3, 6, 8
Total occurrences = 4


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
Brute Force String Matching
1 mark
Q2
Brute Force String Matching
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Brute Force String Matching
1 mark
Q9
Sequential Search
Implement Sequential Search for string data.
Input:
Names:
Anu, Bala, Charan, Deepa, Esha, Farhan
Search Name:
Deepa
Output:
Name found at position 4
Your Code
names = input().split(", ")
target = input().strip()
try:
index = names.index(target)
print(f"\nOutput:\nName found at position {index + 1}")
except ValueError:
print("\nName not found")
Input
Anu, Bala, Charan, Deepa, Esha, Farhan
Deepa
Output
Output:
Name found at position 4
Submitted


SIMATS ENGINEERING
SIMATS Online Programming Lab
DAA PROGRAMMING
Priyadharshini K S
192511248D
- Pattern occurrence positions
Q4
Brute Force String Matching
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Sequential Search
1 mark
Q7
Sequential Search
1 mark
Q8
Brute Force String Matching
1 mark
Q9
Sequential Search
1 mark
Q10
Sequential Search
1 mark
Q11
Sequential Search
1 mark
Your Code
def brute_force_string_matching(text, pattern):
n, m = len(text), len(pattern)
occurrences = []
for i in range(n - m + 1):
match = True
for j in range(m):
if text[i + j] != pattern[j]:
match = False
break
if match:
occurrences.append(i)
print(f"Alignment {i + 1}:")
print(f"Text: {text}")
print(f"Pattern: {' ' * i}{pattern}")
print(f"Match Found at position: {i}\n")
return occurrences
text = "ABCDABCABCDA"
pattern = "ABCDA"
results = brute_force_string_matching(text, pattern)
print(f"Pattern occurrence positions: {results}")
Input
stdin input
Output
Alignment 1:
Text: ABCDABCABCDA
Pattern: ABCDA
Match Found at position: 0
Submitted